

1. **Explain why a document-oriented NoSQL database like MongoDB is better suited for this situation than a relational database. In your answer, address schema flexibility, data nesting, and horizontal scalability — and describe at least one concrete trade-off of choosing MongoDB.**

Ans:- MongoDB is better suited because:

- **Schema flexibility:** MongoDB allows documents to have different fields, making it easy to change or add data without altering a fixed table structure.
 - **Data nesting:** Related data, such as order items, can be stored directly inside an order document, reducing the need for complex JOINS.
 - **Horizontal scalability:** MongoDB can distribute data across multiple servers using sharding, making it suitable for large-scale applications.
 - **Trade-off:** MongoDB can use more storage and may be less suitable when complex transactions and relationships between many tables are required.
2. **Describe the difference between insertOne and insertMany. Write the mongosh commands to create a database quickeats_db, create a collection orders, and insert two orders — one with insertOne and one batch of three orders with insertMany. Explain why insertMany is preferred over multiple insertOne calls for bulk inserts.**

Ans:- (i) insertOne() inserts one document, while **insertMany()** inserts multiple documents at once.

(ii)insertMany? It is faster and more efficient for bulk inserts because it reduces database/network round trips.

3. **Write a MongoDB query using \$in, \$eq, and \$gt to retrieve only matching orders. Also write a query that uses projection to return only customerName, totalAmount, and status, with results sorted by totalAmount descending and limited to 10 documents. Explain what each operator does in this context.**

Ans:-

- **\$in:** Matches any value from a given list.
- **\$eq:** Matches an exact value.
- **\$gt:** Matches values greater than the specified value.
- **Projection:** Returns only required fields.
- **sort(-1):** Sorts descending.
- **limit(10):** Returns maximum 10 documents.

4. Explain the difference between \$set, \$push, and \$addToSet. Then write the updateOne command that: sets status to 'confirmed' and restaurantNote to 'Order accepted', and pushes {event: 'confirmed', time: new Date()} into the statusHistory array. Justify which operator you chose for each field.

Ans:-

- **\$set:** Adds or changes a field.
- **\$push:** Adds an item to an array, allowing duplicates.
- **\$addToSet:** Adds an item to an array only if it doesn't already exist.
- \$set is used for normal fields, while \$push is used to add a new event to the statusHistory array.

5. Write a MongoDB aggregation pipeline using \$match, \$group, \$project, and \$sort to produce this report. Explain what each stage does and why aggregation pipelines are more efficient for this task than reading all documents into Node.js and calculating totals there.

- **Ans:- \$match:** Filters documents.
- **\$group:** Calculates totals.
- **\$project:** Selects/renames output fields.
- **\$sort:** Sorts the report.

Aggregation is more efficient because MongoDB performs filtering and calculations on the database server, avoiding the need to transfer all documents to Node.js.

6. Explain how indexes work in MongoDB and why they reduce read query time. Write the command to create a compound index on {customerId: 1, status: 1}. Then evaluate the trade-off: under what conditions does adding an index hurt performance, and under what conditions is it clearly worth the cost?

Ans:- An **index** is a data structure that helps MongoDB find documents faster without scanning the entire collection.

- Indexes **slow down inserts/updates** and consume extra storage because indexes must also be maintained.
- They are **worth** it when fields are frequently used for searching, filtering, sorting, or querying large collections.
- Too many indexes can hurt write performance.

Section A — Concept Application

```
> use("Foodie_APP")
< switched to db Foodie_APP
> db["Section A"].find()
<
> db.menu.insertOne({
  restaurantId: 101,
  itemName: "Veg Burger",
  price: 149,
  isVeg: true,
  preparationTime: 15,
  addOns: [
    {
      name: "Extra Cheese",
      price: 30
    },
    {
      name: "Extra Patty",
      price: 50
    }
  ]
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a990a920348a65470ee3b92')
}
Foodie_APP> |
```

2.

```
> use("Foodie_APP")
< switched to db Foodie_APP
> db["Section A"].find()
<
> db.menu.insertOne({
  restaurantId: 101,
  itemName: "Veg Burger",
  price: 149,
  isVeg: true,
  preparationTime: 15,
  addOns: [
    {
      name: "Extra Cheese",
      price: 30
    },
    {
      name: "Extra Patty",
      price: 50
    }
  ]
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a990a920348a65470ee3b92')
}
> use quickeats_db
< switched to db quickeats_db
> db.createCollection("orders")
< { ok: 1 }
> db.orders.insertMany([
  {
    customerName: "Rahul Sharma",

    items: [
      {
        name: "Veg Burger",
        quantity: 2,
        price: 149
      },
      {
        name: "French Fries",
        quantity: 1,
        price: 99
      }
    ]
  },

```

```
    totalAmount: 397,

    deliveryAddress: {
      street: "12 MG Road",
      city: "Ahmedabad",
      state: "Gujarat",
      pincode: "380001"
    },

    status: "Pending"
  },
  {
    customerName: "Priya Patel",

    items: [
      {
        name: "Paneer Pizza",
        quantity: 1,
        price: 299
      },
      {
        name: "Cold Drink",
        quantity: 2,
        price: 60
      }
    ],

    totalAmount: 419,

    deliveryAddress: {
      street: "25 Satellite Road",
      city: "Ahmedabad",
      state: "Gujarat",
      pincode: "380015"
    },

    status: "Confirmed"
  },
  {
    customerName: "Amit Shah",

```

```

items: [
  {
    name: "Masala Dosa",
    quantity: 2,
    price: 120
  },
  {
    name: "Idli",
    quantity: 1,
    price: 80
  },
  {
    name: "Mango Lassi",
    quantity: 1,
    price: 90
  }
],

totalAmount: 410,

deliveryAddress: {
  street: "8 CG Road",
  city: "Ahmedabad",
  state: "Gujarat",
  pincode: "380009"
},

status: "Preparing"
},

{
  customerName: "Neha Mehta",

  items: [
    {
      name: "Veg Biryani",
      quantity: 1,
      price: 220
    },
    {
      name: "Raita",
      quantity: 1,
      price: 50
    }
  ]
},

```

```

totalAmount: 270,

deliveryAddress: {
  street: "18 Vastrapur",
  city: "Ahmedabad",
  state: "Gujarat",
  pincode: "380052"
},

status: "Out for Delivery"
},

{
  customerName: "Karan Desai",

  items: [
    {
      name: "Cheese Sandwich",
      quantity: 2,
      price: 130
    },
    {
      name: "Cold Coffee",
      quantity: 2,
      price: 110
    }
  ]
},

totalAmount: 480,

deliveryAddress: {
  street: "45 Paldi Road",
  city: "Ahmedabad",
  state: "Gujarat",
  pincode: "380007"
},

status: "Delivered"
}
])

```

```

< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a990be80348a65470ee3b93'),
    '1': ObjectId('6a990be80348a65470ee3b94'),
    '2': ObjectId('6a990be80348a65470ee3b95'),
    '3': ObjectId('6a990be80348a65470ee3b96'),
    '4': ObjectId('6a990be80348a65470ee3b97')
  }
}
> db.orders.find()
< {
  _id: ObjectId('6a990be80348a65470ee3b93'),
  customerName: 'Rahul Sharma',
  items: [
    {
      name: 'Veg Burger',
      quantity: 2,
      price: 149
    },
    {
      name: 'French Fries',
      quantity: 1,
      price: 99
    }
  ],
  totalAmount: 397,
  deliveryAddress: {
    street: '12 MG Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380001'
  },
  status: 'Pending'
}
{
  _id: ObjectId('6a990be80348a65470ee3b94'),
  customerName: 'Priya Patel',
  items: [
    {
      name: 'Paneer Pizza',
      quantity: 1,
      price: 299
    },
    {

```

```

    {
      name: 'Cold Drink',
      quantity: 2,
      price: 60
    }
  ],
  totalAmount: 419,
  deliveryAddress: {
    street: '25 Satellite Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380015'
  },
  status: 'Confirmed'
}
{
  _id: ObjectId('6a990be80348a65470ee3b95'),
  customerName: 'Amit Shah',
  items: [
    {
      name: 'Masala Dosa',
      quantity: 2,
      price: 120
    },
    {
      name: 'Idli',
      quantity: 1,
      price: 80
    },
    {
      name: 'Mango Lassi',
      quantity: 1,
      price: 90
    }
  ],
  totalAmount: 410,
  deliveryAddress: {
    street: '8 CG Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380009'
  },
  status: 'Preparing'
}
{

```

```

    _id: ObjectId('6a990be80348a65470ee3b96'),
    customerName: 'Neha Mehta',
    items: [
      {
        name: 'Veg Biryani',
        quantity: 1,
        price: 220
      },
      {
        name: 'Raita',
        quantity: 1,
        price: 50
      }
    ],
    totalAmount: 270,
    deliveryAddress: {
      street: '18 Vastrapur',
      city: 'Ahmedabad',
      state: 'Gujarat',
      pincode: '380052'
    },
    status: 'Out for Delivery'
  }
  {
    _id: ObjectId('6a990be80348a65470ee3b97'),
    customerName: 'Karan Desai',
    items: [
      {
        name: 'Cheese Sandwich',
        quantity: 2,
        price: 130
      },
      {
        name: 'Cold Coffee',
        quantity: 2,
        price: 110
      }
    ],
    totalAmount: 480,
    deliveryAddress: {
      street: '45 Paldi Road',
      city: 'Ahmedabad',
      state: 'Gujarat',
      pincode: '380007'
    },

```

```

    status: 'Delivered'
  }
  > db.orders.find().pretty()
< [
  {
    _id: ObjectId('6a990be80348a65470ee3b93'),
    customerName: 'Rahul Sharma',
    items: [
      {
        name: 'Veg Burger',
        quantity: 2,
        price: 149
      },
      {
        name: 'French Fries',
        quantity: 1,
        price: 99
      }
    ],
    totalAmount: 397,
    deliveryAddress: {
      street: '12 MG Road',
      city: 'Ahmedabad',
      state: 'Gujarat',
      pincode: '380001'
    },
    status: 'Pending'
  }
  {
    _id: ObjectId('6a990be80348a65470ee3b94'),
    customerName: 'Priya Patel',
    items: [
      {
        name: 'Paneer Pizza',
        quantity: 1,
        price: 299
      },
      {
        name: 'Cold Drink',
        quantity: 2,
        price: 60
      }
    ],
    totalAmount: 419,
    deliveryAddress: {
      street: '25 Satellite Road',

```

```

    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380015'
  },
  status: 'Confirmed'
}
{
  _id: ObjectId('6a990be80348a65470ee3b95'),
  customerName: 'Amit Shah',
  items: [
    {
      name: 'Masala Dosa',
      quantity: 2,
      price: 120
    },
    {
      name: 'Idli',
      quantity: 1,
      price: 80
    },
    {
      name: 'Mango Lassi',
      quantity: 1,
      price: 90
    }
  ],
  totalAmount: 410,
  deliveryAddress: {
    street: '8 CG Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380009'
  },
  status: 'Preparing'
}
{
  _id: ObjectId('6a990be80348a65470ee3b96'),
  customerName: 'Neha Mehta',
  items: [
    {
      name: 'Veg Biryani',
      quantity: 1,
      price: 220
    },
    {

```

```

    {
      name: 'Raita',
      quantity: 1,
      price: 50
    }
  ],
  totalAmount: 270,
  deliveryAddress: {
    street: '18 Vastrapur',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380052'
  },
  status: 'Out for Delivery'
}
{
  _id: ObjectId('6a990be80348a65470ee3b97'),
  customerName: 'Karan Desai',
  items: [
    {
      name: 'Cheese Sandwich',
      quantity: 2,
      price: 130
    },
    {
      name: 'Cold Coffee',
      quantity: 2,
      price: 110
    }
  ],
  totalAmount: 480,
  deliveryAddress: {
    street: '45 Paldi Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380007'
  },
  status: 'Delivered'
}

```

```

> db.orders.find({ status: "Pending" })
< {
  _id: ObjectId('6a990be80348a65470ee3b93'),
  customerName: 'Rahul Sharma',
  items: [
    {
      name: 'Veg Burger',
      quantity: 2,
      price: 149
    },
    {
      name: 'French Fries',
      quantity: 1,
      price: 99
    }
  ],
  totalAmount: 397,
  deliveryAddress: {
    street: '12 MG Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380001'
  },
  status: 'Pending'
}

```

```

> db.orders.find({ status: "Delivered" })
< {
  _id: ObjectId('6a990be80348a65470ee3b97'),
  customerName: 'Karan Desai',
  items: [
    {
      name: 'Cheese Sandwich',
      quantity: 2,
      price: 130
    },
    {
      name: 'Cold Coffee',
      quantity: 2,
      price: 110
    }
  ],
  totalAmount: 480,
  deliveryAddress: {
    street: '45 Paldi Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380007'
  },
  status: 'Delivered'
}

```

```

> db.orders.find({ customerName: "Rahul Sharma" })
< {
  _id: ObjectId('6a990be80348a65470ee3b93'),
  customerName: 'Rahul Sharma',
  items: [
    {
      name: 'Veg Burger',
      quantity: 2,
      price: 149
    },
    {
      name: 'French Fries',
      quantity: 1,
      price: 99
    }
  ],
  totalAmount: 397,
  deliveryAddress: {
    street: '12 MG Road',
    city: 'Ahmedabad',
    state: 'Gujarat',
    pincode: '380001'
  },
  status: 'Pending'
}
> db.orders.updateOne(
  { customerName: "Rahul Sharma" },
  { $set: { status: "Confirmed" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.orders.countDocuments()
< 5
quickeats_db>

```

3. If only required fields are needed

```
1. db.orders.find({  
  status: { $in: ["Pending", "Confirmed"] },  
  customerName: "Rahul Sharma",  
  totalAmount: { $gt: 500 }  
})
```

```
2. db.orders.createIndex({  
  customerName: 1,  
  status: 1,  
  totalAmount: 1  
})
```

```
3. db.orders.find(  
  {  
    status: { $in: ["Pending", "Confirmed"] },  
    customerName: "Rahul Sharma",  
    totalAmount: { $gt: 500 }  
  },  
  {  
    customerName: 1,  
    items: 1,  
    totalAmount: 1,  
    deliveryAddress: 1,  
    status: 1
```

```
}  
)
```

Final recommended query:

```
db.orders.find(  
  status: { $in: ["Pending", "Confirmed"] },  
  customerName: "Rahul Sharma",  
  totalAmount: { $gt: 500 }  
)
```

```
db.orders.createIndex(  
  customerName: 1,  
  status: 1,  
  totalAmount: 1  
)
```

4. use quickeats_db

```
db.orders.updateOne(  
  { _id: ObjectId("64f123456789abcdef123456") },  
  
  {  
    // Update existing status and add/update restaurantNote  
    $set: {  
      status: "confirmed",  
      restaurantNote: "Restaurant has accepted the order."    }  
  }  
)
```

```
    },

    // Add a new entry to statusHistory
    $push: {
      statusHistory: {
        status: "confirmed",
        message: "Restaurant accepted the order.",
        updatedAt: new Date()
      }
    }
  }
)

{
  _id: ObjectId("64f123456789abcdef123456"),
  customerName: "Rahul Sharma",
  totalAmount: 650,
  status: "pending",

  statusHistory: [
    {
      status: "pending",
      message: "Order placed.",
      updatedAt: ISODate("2026-09-03T10:00:00Z")
    }
  ]
}
```

```
}  
  
{  
  _id: ObjectId("64f123456789abcdef123456"),  
  customerName: "Rahul Sharma",  
  totalAmount: 650,  
  status: "confirmed",  
  
  restaurantNote: "Restaurant has accepted the order.",  
  
  statusHistory: [  
    {  
      status: "pending",  
      message: "Order placed.",  
      updatedAt: ISODate("2026-09-03T10:00:00Z")  
    },  
    {  
      status: "confirmed",  
      message: "Restaurant accepted the order.",  
      updatedAt: ISODate("2026-09-03T10:15:00Z")  
    }  
  ]  
}
```

```
db.orders.updateOne(  
  { _id: ObjectId("64f123456789abcdef123456") },  
  {
```

```
$addToSet: {  
  tags: "priority"  
}  
}
```

5. 1. \$match

Filters the collection before calculations:

```
{  
  $match: {  
    status: "completed",  
    orderDate: {  
      $gte: ISODate("2026-08-31T00:00:00Z"),  
      $lt: ISODate("2026-09-07T00:00:00Z")  
    }  
  }  
}
```

2. \$group

Groups orders by restaurant and calculates both metrics:

```
{  
  $group: {  
    _id: "$restaurantId",  
    totalRevenue: { $sum: "$totalAmount" },  
    completedOrders: { $sum: 1 }  
  }  
}
```

3. \$sort

Ranks restaurants by revenue:

```
{  
  $sort: {  
    totalRevenue: -1  
  }  
}
```

Recommended index

```
db.orders.createIndex({  
  status: 1,  
  orderDate: 1,  
  restaurantId: 1  
})
```

6.

1. Current Query

```
db.orders.find({  
  customerId: 101,  
  status: "pending"  
})
```

2. Create a Compound Index

```
use quickeats_db
```

```
db.orders.createIndex({  
  customerId: 1,  
  status: 1  
})
```

3. Check Index Usage

```
db.orders.find({  
  customerId: 101,  
  status: "pending"  
}).explain("executionStats")
```

Section B — Practical Coding Tasks

Task 1: Set Up the QuickEats Restaurant Database

→ 1. Create database and collection

```
use quickeats_db  
db.createCollection("restaurants")
```

2. Insert 5 restaurants using insertMany()

```
db.restaurants.insertMany([  
  {  
    name: "Spice Garden",  
    cuisine: "Indian",  
    city: "Surat",  
    rating: 4.5,  
    isOpen: true,  
    menu: [  
      {  
        itemName: "Paneer Tikka",  
        price: 220  
      },  
      {  
        itemName: "Veg Biryani",
```

```
    price: 180
  }
]
},
```

```
{
  name: "Pizza Palace",
  cuisine: "Italian",
  city: "Ahmedabad",
  rating: 4.3,
  isOpen: true,
  menu: [
    {
      itemName: "Margherita Pizza",
      price: 250
    },
    {
      itemName: "Farmhouse Pizza",
      price: 350
    }
  ]
},
```

```
{
  name: "South Indian Hub",
  cuisine: "South Indian",
```

```
city: "Vadodara",
rating: 4.6,
isOpen: true,
menu: [
  {
    itemName: "Masala Dosa",
    price: 120
  },
  {
    itemName: "Idli Sambar",
    price: 90
  }
]
```

```
{
  name: "Burger House",
  cuisine: "Fast Food",
  city: "Surat",
  rating: 4.1,
  isOpen: false,
  menu: [
    {
      itemName: "Veg Burger",
      price: 150
    },

```

```
{
  itemName: "French Fries",
  price: 100
}
],
},
```

```
{
  name: "Royal Thali",
  cuisine: "Gujarati",
  city: "Rajkot",
  rating: 4.8,
  isOpen: true,
  menu: [
    {
      itemName: "Gujarati Thali",
      price: 280
    },
    {
      itemName: "Basundi",
      price: 120
    }
  ]
}
D)
```

3. Display all inserted documents

```
db.restaurants.find()
```

```
db.restaurants.find().pretty()
```

4. Verify the number of documents

```
db.restaurants.countDocuments()
```

5

Task 2: Query and Filter Orders Using Operators

→ 1. Select database and create orders collection

use quickeats_db

```
db.createCollection("orders")
```

2. Insert 8 orders using insertMany()

```
db.orders.insertMany([
```

```
{
```

```
  customerId: "C001",
```

```
  restaurantName: "Spice Garden
```

Task 3: Update Order Records Using Update Operators

→ use quickeats_db

// 1. Change one pending order to confirmed

```
db.orders.updateOne(
```

```
  { status: "pending" },
```

```
{
```

```
  $set: {
```

```
    status: "confirmed",
```

```
        confirmedAt: new Date()
    }
}
)
```

```
// Verify updateOne
db.orders.find({
  status: "confirmed"
}).pretty()
```

// 2. Change all confirmed orders to delivered

```
db.orders.updateMany(
  { status: "confirmed" },
  {
    $set: {
      status: "delivered"
    }
  }
)
```

```
// Verify updateMany
db.orders.find({
  status: "delivered"
}).pretty()
```

// 3. Add dispatched event to deliveryLog

```
db.orders.updateOne(
  { customerId: "C001" },
  {
    $push: {
      deliveryLog: {
        event: "dispatched",
        time: new Date()
      }
    }
  }
)
```

// Verify \$push

```
db.orders.find({
  customerId: "C001"
}).pretty()
```

// 4. Remove confirmedAt

```
db.orders.updateOne(
  { customerId: "C001" },
  {
    $unset: {
      confirmedAt: ""
    }
  }
)
```

```
}  
}  
)
```

// Verify Sunset

```
db.orders.find({  
  customerId: "C001"  
}).pretty()
```

Task 4: Restaurant Revenue Report Using Aggregation

→

1. Aggregation Pipeline

use quickeats_db

```
db.orders.aggregate([
```

// 1. Filter only delivered orders

```
{  
  $match: {  
    status: "delivered"  
  }  
},
```

// 2. Group orders by restaurant

```
{  
  $group: {  
    _id: "$restaurantName",
```

```
    totalRevenue: {
      $sum: "$totalAmount"
    },

    orderCount: {
      $sum: 1
    }
  }
},

// 3. Shape the output
{
  $project: {
    _id: 0,
    restaurantName: "$_id",
    totalRevenue: 1,
    orderCount: 1
  }
},

// 4. Sort by revenue, highest first
{
  $sort: {
    totalRevenue: -1
  }
}
```

```
},
```

```
// 5. Display only top 3 restaurants
```

```
{
```

```
  $limit: 3
```

```
}
```

```
D
```

Example result

```
[
```

```
{
```

```
  restaurantName: "Spice Garden",
```

```
  totalRevenue: 1850,
```

```
  orderCount: 5
```

```
},
```

```
{
```

```
  restaurantName: "Pizza Palace",
```

```
  totalRevenue: 1420,
```

```
  orderCount: 4
```

```
},
```

```
{
```

```
  restaurantName: "Royal Thali",
```

```
  totalRevenue: 980,
```

```
  orderCount: 3
```

```
}
```

]

4. What each stage does

{

\$match: {

status: "delivered"

}

}

{

\$group: {

_id: "\$restaurantName",

totalRevenue: { \$sum: "\$totalAmount" },

orderCount: { \$sum: 1 }

}

}

{

\$project: {

_id: 0,

restaurantName: "\$_id",

totalRevenue: 1,

orderCount: 1

}

}

{

\$sort: {

totalRevenue: -1

}

```
}  
  
{  
  $limit: 3  
}
```

Section C — Mini Capstone Project

Step 1: Create Database

→use quickeats_db

Step 2: Create Collections

db.createCollection("restaurants")

db.createCollection("customers")

db.createCollection("orders")

Step 3: Insert Documents

A. Restaurants Collection

```
db.restaurants.insertMany([  
{  
  name: "Spice Garden",  
  cuisine: "Indian",  
  city: "Surat",  
  rating: 4.5,  
  isOpen: true,  
  menu: [  
    { itemName: "Paneer Tikka", price: 220 },  
    { itemName: "Veg Biryani", price: 180 }  
  ]  
},
```

```
{
  name: "Pizza Palace",
  cuisine: "Italian",
  city: "Ahmedabad",
  rating: 4.3,
  isOpen: true,
  menu: [
    { itemName: "Margherita Pizza", price: 250 },
    { itemName: "Farmhouse Pizza", price: 350 }
  ]
},
{
  name: "Burger House",
  cuisine: "Fast Food",
  city: "Surat",
  rating: 4.1,
  isOpen: false,
  menu: [
    { itemName: "Veg Burger", price: 150 },
    { itemName: "French Fries", price: 100 }
  ]
},
{
  name: "Royal Thali",
  cuisine: "Gujarati",
  city: "Rajkot",
```

```
    rating: 4.8,
    isOpen: true,
    menu: [
      { itemName: "Gujarati Thali", price: 280 },
      { itemName: "Basundi", price: 120 }
    ]
  },
  {
    name: "South Indian Hub",
    cuisine: "South Indian",
    city: "Vadodara",
    rating: 4.6,
    isOpen: true,
    menu: [
      { itemName: "Masala Dosa", price: 120 },
      { itemName: "Idli", price: 80 }
    ]
  }
]
```

B. Customers Collection

```
db.customers.insertMany([
  {
    customerId: "C001",
    name: "Rahul Sharma",
    city: "Surat",
    loyaltyPoints: 120,
```

```
addresses: [  
  {  
    type: "Home",  
    address: "Adajan, Surat"  
  }  
]  
,  
{  
  customerId: "C002",  
  name: "Priya Patel",  
  city: "Ahmedabad",  
  loyaltyPoints: 80,  
  addresses: [  
    {  
      type: "Home",  
      address: "Satellite, Ahmedabad"  
    }  
  ]  
},  
{  
  customerId: "C003",  
  name: "Amit Shah",  
  city: "Rajkot",  
  loyaltyPoints: 150,  
  addresses: [  
    {
```

```
        type: "Office",
        address: "Kalawad Road, Rajkot"
    }
]
},
{
    customerId: "C004",
    name: "Neha Mehta",
    city: "Surat",
    loyaltyPoints: 95,
    addresses: [
        {
            type: "Home",
            address: "Vesu, Surat"
        }
    ]
},
{
    customerId: "C005",
    name: "Karan Desai",
    city: "Vadodara",
    loyaltyPoints: 60,
    addresses: [
        {
            type: "Home",
            address: "Alkapuri, Vadodara"
```

```
    }  
  ]  
}  
D)
```

C. Orders Collection

```
db.orders.insertMany([  
  {  
    customerId: "C001",  
    restaurantName: "Spice Garden",  
    items: [  
      { itemName: "Paneer Tikka", qty: 1, price: 220 },  
      { itemName: "Veg Biryani", qty: 1, price: 180 }  
    ],  
    totalAmount: 400,  
    status: "delivered",  
    orderDate: ISODate("2026-09-01T10:00:00Z")  
  },  
  {  
    customerId: "C002",  
    restaurantName: "Pizza Palace",  
    items: [  
      { itemName: "Farmhouse Pizza", qty: 2, price: 350 }  
    ],  
    totalAmount: 700,  
    status: "pending",  
    orderDate: ISODate("2026-09-01T12:00:00Z")  
  }  
])
```

```
},  
{  
  customerId: "C003",  
  restaurantName: "Royal Thali",  
  items: [  
    { itemName: "Gujarati Thali", qty: 2, price: 280 }  
  ],  
  totalAmount: 560,  
  status: "confirmed",  
  orderDate: ISODate("2026-09-02T13:00:00Z")  
},  
{  
  customerId: "C004",  
  restaurantName: "South Indian Hub",  
  items: [  
    { itemName: "Masala Dosa", qty: 3, price: 120 }  
  ],  
  totalAmount: 360,  
  status: "delivered",  
  orderDate: ISODate("2026-09-03T09:00:00Z")  
},  
{  
  customerId: "C005",  
  restaurantName: "Spice Garden",  
  items: [  
    { itemName: "Veg Biryani", qty: 2, price: 180 }
```

```
    ],  
    totalAmount: 360,  
    status: "delivered",  
    orderDate: ISODate("2026-09-03T11:00:00Z")  
  }  
)  

```

Step 4: Read Queries

Restaurants with rating > 4

```
db.restaurants.find(  
  { rating: { $gt: 4 } },  
  { _id: 0, name: 1, rating: 1 }  
)
```

Orders with pending or confirmed status

```
db.orders.find(  
  { status: { $in: ["pending", "confirmed"] } },  
  { _id: 0, customerId: 1, totalAmount: 1, status: 1 }  
)
```

Customers from Surat

```
db.customers.find(  
  { city: { $eq: "Surat" } },  
  { _id: 0, name: 1, city: 1 }  
)
```

Step 5: Update Operations

Update order status

```
db.orders.updateOne(  
  { customerId: "C002" },
```

```
{
  $set: {
    status: "confirmed"
  }
}
)
```

Add delivery log

```
db.orders.updateOne(
  { customerId: "C001" },
  {
    $push: {
      deliveryLog: {
        event: "dispatched",
        time: new Date()
      }
    }
  }
)
)
```

Increment loyalty points

```
db.customers.updateOne(
  { customerId: "C001" },
  {
    $inc: {
      loyaltyPoints: 20
    }
  }
)
```

)

Step 6: Aggregation Pipeline

```
db.orders.aggregate([
{
  $match: {
    status: "delivered"
  }
},
{
  $group: {
    _id: "$restaurantName",
    totalRevenue: {
      $sum: "$totalAmount"
    },
    orderCount: {
      $sum: 1
    }
  }
},
{
  $project: {
    _id: 0,
    restaurantName: "$_id",
    totalRevenue: 1,
    orderCount: 1
  }
}]
```

```
    }  
  },  
  {  
    $sort: {  
      totalRevenue: -1  
    }  
  }  
}  
D)
```

Step 7: Create Compound Index

```
db.orders.createIndex({  
  customerId: 1,  
  status: 1  
})
```

Step 8: Explain Before Index

```
db.orders.find({  
  customerId: "C001",  
  status: "delivered"  
}).explain("executionStats")
```

Look for:

COLLSCAN

totalDocsExamined: 500000

Step 9: Explain After Index

```
db.orders.find({
```

```
    customerId: "C001",  
    status: "delivered"  
}).explain("executionStats")
```

Expected:

IXSCAN

totalDocsExamined: 1